PROTEA HEIGHTS ACADEMY · IT

PHA WiFi setup guide

Setting up the Linux server and the Ruckus vSZ, from an empty server to the first pilot logins with school Google accounts.

IN THIS GUIDE

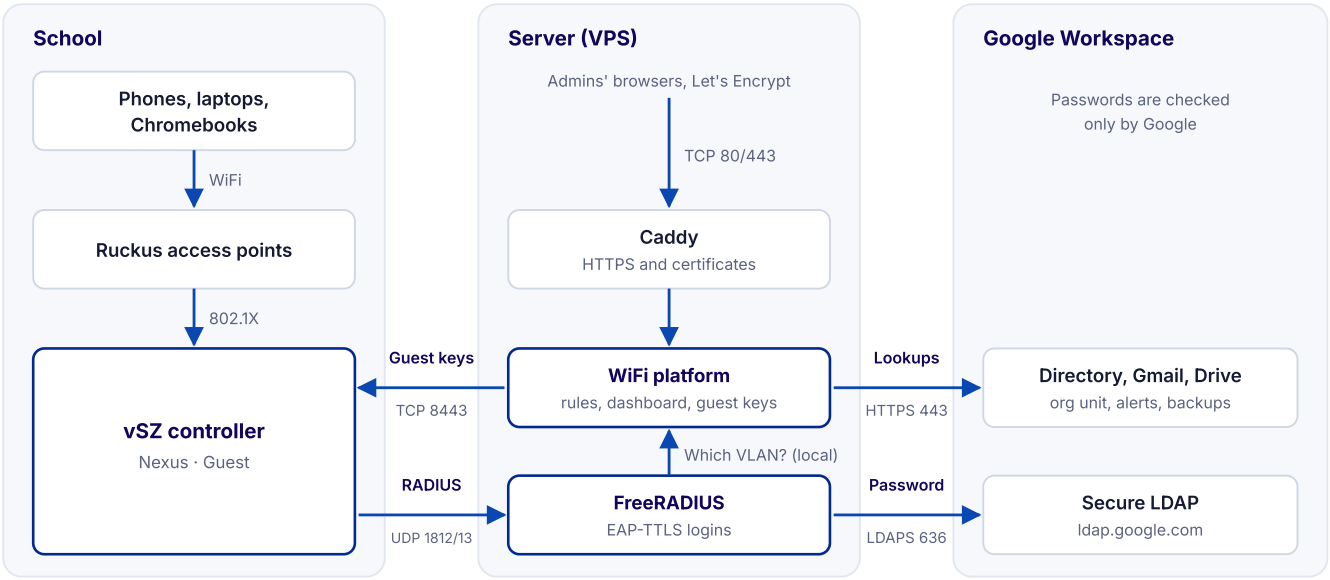
- What to collect and decide before you start
- Preparing an Ubuntu server: SSH, firewall, Docker
- Google Workspace: Secure LDAP, sign-in, directory
- Ruckus vSZ: RADIUS services, Nexus, Guest, API account
- The school firewall: what to open, and to whom
- Finishing the server, then testing a real login
- Setting up devices, and the two-week pilot
- Updates, backups, restores and troubleshooting

For	Software	Date	Source
School IT staff	PHA WiFi 0.2.0	5 November 2026	docs/setup-guide.html in the repo

OVERVIEW

How it fits together

People join **Nexus** with their school Google email and password. The password is checked by Google, and the platform puts each person on the VLAN for their Google org unit. Visitors use **Guest** with keys that admins make in the dashboard.



One login: the device sends the email and password inside a TLS tunnel to FreeRADIUS. FreeRADIUS checks the password with Google Secure LDAP. Only if it's right does it ask the platform whether this person may connect, and on which VLAN. The vSZ then puts the device on that VLAN.

Contents

1	**Plan**	What to collect and decide first	3
2	**Prepare the Linux server**	Ubuntu, SSH, firewall, Docker, the code	4
3	**Google Workspace**	Secure LDAP, admin sign-in, directory, alerts	6
4	**Ruckus vSZ**	RADIUS services, both WLANs, API account, school firewall	8
5	**Finish the server and test**	Settings, secrets, start, a real login	11
6	**Devices and the pilot**	Setting up each kind of device, the go-live checklist	14
7	**Looking after it**	Updates, backups, restores, renewals	16
8	**When something doesn't work**	Symptoms, causes and fixes	18
A	**Ports and files**	Quick reference	20

Why the parts are in this order
The platform won't start until it has the vSZ API details, and the vSZ needs the server's address and shared secret. So you prepare the server first (part 2), set up Google and the vSZ (parts 3 and 4), then come back to finish the server (part 5).

PART 1

Plan

Collect these before you start. Filling in the sheet below saves going back and forth between the server, Google and the vSZ.

You'll need

- **A Linux VPS** with a fixed public IP address: Ubuntu Server 24.04 LTS, 1 vCPU, 1 GB RAM and 20 GB of disk is plenty.
- **DNS access** for `phahs.org.za`, to add two records.
- **A super admin account** in the Google Admin console, and access to Google Cloud for the same domain.
- **Admin access to the vSZ** and to the school's firewall or router.
- **VLANs for staff, learners and guests** that already reach every access point's switch port, each with a DHCP scope.
- A PC with an SSH client. Windows 10 and 11 include `ssh` and `scp` in PowerShell.

Worksheet

Item	Example	Yours
Server public IP	198.51.100.20	
School public IP: where the vSZ's traffic appears on the internet	203.0.113.10	
vSZ address the server uses for the API (port 8443)	203.0.113.10	
vSZ software version	7.0	
Your own public IP(s), for SSH	203.0.113.10	
Staff VLAN	20	
Learner VLAN	30	
Guest VLAN	50	
VLAN while the platform is down, or none	none	
Google org unit for staff	/Staff	
Google org unit for learners	/Students	
Dashboard admins	llotter@phahs.org.za	
Mailbox alerts are sent from	wifi-alerts@phahs.org.za	
People who get alerts	llotter@phahs.org.za	

Secrets never go on this sheet
The RADIUS shared secret, the platform token, the vSZ API password, the Google OAuth client secret, and the Google key and certificate files go in a password manager, never on paper, in email or in chat. Part 2 shows how to make the first two.

Decide: what happens if the platform is down

If the platform stops while FreeRADIUS and Google are fine, FreeRADIUS follows `FALLBACK_VLAN`:

- **Empty** (the default): no new logins until the platform is back. Devices already connected stay connected.
- **A VLAN number**: anyone with a correct Google password gets on to that VLAN, and blocks and rules are skipped until the platform is back. If you choose this, use a restricted learner-style VLAN, never the staff VLAN.

PART 2

Prepare the Linux server

This part sets up an Ubuntu 24.04 server with key-only SSH, a firewall, automatic security updates and Docker, then puts the code and the settings folders in place.

2.1 Create the server and its DNS names

1. Create the VPS with **Ubuntu Server 24.04 LTS**. If the provider has its own firewall, allow the same ports as in 2.4.
2. Add two DNS **A records** pointing at the server's public IP:
   - `wifi.phahs.org.za`: the admin dashboard.
   - `radius.phahs.org.za`: the certificate devices check when they log in.
3. Check both from your PC. Each should print the server's IP:

ON YOUR PC (POWERSHELL)

```
nslookup wifi.phahs.org.za
nslookup radius.phahs.org.za
```

2.2 First login, updates and an admin user

Log in as `root` (or the user your provider made). These examples call the admin user `itadmin`.

ON THE SERVER, AS ROOT

```
apt update && apt full-upgrade -y
timedatectl set-timezone Africa/Johannesburg
apt install -y git unattended-upgrades
adduser itadmin
usermod -aG sudo itadmin
```

Copy your SSH key to the new user. If you don't have a key yet, run `ssh-keygen -t ed25519` on your PC first.

ON YOUR PC (POWERSHELL)

```
type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh itadmin@SERVER_IP "mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys &&
chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys"
```

Log in as `itadmin` with the key. Then turn off password logins and root logins:

ON THE SERVER, AS ITADMIN

```
sudo tee /etc/ssh/sshd_config.d/10-pha.conf >/dev/null <<'EOF'
PasswordAuthentication no
PermitRootLogin no
EOF
sudo systemctl restart ssh
sudo dpkg-reconfigure -plow unattended-upgrades    # answer Yes
```

> **Keep your current session open**
> Before you close it, open a second SSH session as `itadmin` and check it works. If it doesn't, fix it from the first session.

2.3 Install Docker

These are Docker's own instructions for Ubuntu. They install Docker Engine and the Compose plugin.

```
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo docker run --rm hello-world       # prints "Hello from Docker!"
sudo docker compose version
```

This guide runs Docker with `sudo`. Adding `itadmin` to the `docker` group would save typing, but that group is equivalent to root.

2.4 Firewall

Only SSH from your own addresses, the web ports, and RADIUS from the school are let in. Allow SSH **before** turning the firewall on.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from YOUR_IP to any port 22 proto tcp
sudo ufw allow 80,443/tcp
sudo ufw allow from SCHOOL_IP to any port 1812:1813 proto udp
sudo ufw enable
sudo ufw status verbose
```

- Repeat the SSH line for each address you manage the server from. If your own IP changes often, use `sudo ufw allow 22/tcp` instead: SSH accepts keys only, so that's still safe.
- **SCHOOL_IP** is the address RADIUS from the vSZ arrives from (see 4.7). Part 8 shows how to read it from the logs if you're unsure.
- All three containers use the host's network directly, without published ports, so these `ufw` rules apply to them as normal. Docker's usual habit of skipping `ufw` for published ports doesn't come into it.

2.5 Get the code onto the server

The code is public on GitHub at `github.com/pha-llotter/wifi-platform`, so the server can download it without a key or password. It lives in `/opt/wifi-portal`.

ON THE SERVER

```
sudo mkdir /opt/wifi-portal && sudo chown $USER: /opt/wifi-portal
git clone https://github.com/pha-llotter/wifi-platform.git /opt/wifi-portal
```

The repo holds only code and examples. Every real setting and secret lives in `/etc/wifi-portal` on the server and never goes into Git.

2.6 Settings folders and the two secrets

```
sudo mkdir -p /etc/wifi-portal/google-ldap /var/log/wifi-portal /var/lib/wifi-portal
sudo cp /opt/wifi-portal/config.example.json /etc/wifi-portal/config.json
sudo cp /opt/wifi-portal/deploy/freeradius/radius.env.example /etc/wifi-portal/radius.env
sudo chown 1000 /var/log/wifi-portal /var/lib/wifi-portal
openssl rand -hex 32     # 1: the RADIUS shared secret (vSZ ⇔ FreeRADIUS)
openssl rand -hex 32     # 2: the platform token (FreeRADIUS ⇔ platform)
```

Save both values in your password manager, labelled. The first goes on the vSZ (part 4) and in `radius.env`. The second goes in `radius.env` and `config.json` (part 5).

> **Stop here for now**
> Don't start the containers yet. The platform needs the Google and vSZ details first. Carry on with part 3.

PART 3

Google Workspace

Education Fundamentals includes everything here. You'll end up with three things for the server: the Secure LDAP certificate and key, an OAuth client for the dashboard sign-in, and a service account key for the directory, alerts and backups.

3.1 Secure LDAP client: checks passwords

1. In the **Admin console** go to **Apps** → **LDAP** → **Add client**. Name it `WiFi RADIUS`.
2. Access permissions, both for the **whole domain** (or every org unit that should use the WiFi):
   - **Verify user credentials**
   - **Read user information**
3. **Download the certificate.** The zip holds a `.crt` and a `.key` file. Rename them `ldap-client.crt` and `ldap-client.key`.
4. Set the client's **Service status** to **On**. It can take a few minutes, and sometimes longer, before Google accepts the certificate.
5. Note the certificate's **expiry date**, shown on the client's page, and put a reminder in the calendar a month before it.

Leave **access credentials** (a username and password for the LDAP client) alone for now: the certificate is enough. Part 8 covers the case where they're needed.

3.2 OAuth client: admin sign-in to the dashboard

1. In **Google Cloud console** create a project, for example `PHA WiFi`.
2. **Google Auth Platform** → **Audience**: set it to **Internal**, so only school accounts can sign in.
3. **Clients** → **Create client**: type **Web application**, authorised redirect URI `https://wifi.phahs.org.za/admin/oauth/callback`.
4. Save the **client ID** and **client secret** in the password manager.

3.3 Service account: directory, alerts and backups

1. In the same Cloud project, enable the **Admin SDK API**. Enable the **Gmail API** for alert emails and the **Google Drive API** for off-site backups.
2. **IAM & Admin** → **Service accounts** → **Create**, with no roles. Open it, then **Keys** → **Add key** → **JSON**. Save the file as `service-account.json`.
3. Copy the service account's **client ID** (a long number).
4. In the **Admin console**: **Security** → **Access and data control** → **API controls** → **Domain-wide delegation** → **Add new**. Paste the client ID, and these scopes, comma-separated:

```
https://www.googleapis.com/auth/admin.directory.user.readonly,https://www.googleapis.com/auth/gmail.send,https://www.googleapis.com/auth/drive.file
```

- `admin.directory.user.readonly` reads org units and whether an account is suspended. It's needed.
- `gmail.send` sends alert emails. Leave it out if you don't want alerts.
- `drive.file` uploads nightly backups. It can only see files it uploaded itself. Leave it out if you don't want Drive backups.

> **Your organisation may block service account keys**
> If Google Cloud refuses to make a JSON key, the organisation policy `iam.disableServiceAccountKeyCreation` is on. A Cloud organisation admin can turn it off for this one project.

3.4 Alert mailbox and backup folder (optional)

- **Alerts** are sent as a real Workspace user, for example `wifi-alerts@phahs.org.za`. The service account can send as that user but can't read its mail.
- **Backups**: make a **Shared Drive**, for example `WiFi backups`. Add the alert mailbox user as **Content manager**, make a folder in it, and copy the last part of the folder's URL. That's the folder ID.

3.5 Check the org units

In **Directory → Organizational units**, check the exact paths for staff and learners, for example `/Staff` and `/Students`, and fill them in on the worksheet. A rule matches its org unit and everything under it.

3.6 Copy the files to the server

ON YOUR PC (POWERSHELL), IN THE FOLDER WITH THE THREE FILES

```
scp ldap-client.crt ldap-client.key service-account.json itadmin@SERVER_IP:~
```

ON THE SERVER

```
sudo mv ~/ldap-client.crt ~/ldap-client.key /etc/wifi-portal/google-ldap/
sudo mv ~/service-account.json /etc/wifi-portal/
```

Then delete the copies on your PC, keeping them only in the password manager. Part 5 sets the permissions.

PART 4

Ruckus vSZ

You'll create two RADIUS services that point at the server, the Nexus WLAN with dynamic VLANs, the Guest WLAN with Dynamic PSK, and an API account for guest keys. Then you'll open the school firewall.

> **Menu names**
> The paths below are for SmartZone 6.x and 7.x. In 5.x, the RADIUS services are under **Services & Profiles** → **Authentication** and → **Accounting**. The field names are the same.

4.1 Before you start

- The staff, learner and guest VLANs are tagged on every access point's switch port, and each has a DHCP scope.
- You have the server's public IP and the **RADIUS shared secret** from 2.6.

Use **proxy** mode, where the controller sends RADIUS for the APs, for both services. All RADIUS then comes from the vSZ, so FreeRADIUS only has to trust one address, and the APs' own addresses don't matter.

4.2 RADIUS authentication service

Security → Authentication → Proxy (SZ Authenticator) → Create

Field	Value
Name	`Nexus RADIUS`
Service protocol	RADIUS
Primary server: IP	The server's public IP
Port	`1812`
Shared secret	The RADIUS shared secret from 2.6
Secondary server	Leave off

> **The vSZ's "Test AAA" button will say it failed**
> That test sends a plain username and password. This server refuses anything that isn't a real 802.1X login, on purpose. Use `test-login.sh` on the server instead (5.7), and a real phone (5.9).

4.3 RADIUS accounting service

Security → Accounting → Proxy → Create. Name it `Nexus accounting`, with the same server IP, port `1813` and the same shared secret. This is what feeds the **Usage** page.

4.4 WLAN: Nexus (school accounts)

Network → Wireless → Wireless LANs, select the school's zone, **Create**.

Setting	Value
Name and SSID	`Nexus`. It must match `radius.ssid` in config.json.
Authentication type	Standard usage
Method	**802.1X EAP**
Encryption	WPA2, AES. WPA2/WPA3 mixed also works.
Authentication service	Tick **Use the controller as proxy**, choose `Nexus RADIUS`
Accounting service	Tick **Use the controller as proxy**, choose `Nexus accounting`, interim updates every **10 minutes**

Setting	Value
Access VLAN	Your most restricted VLAN, for example the learner VLAN `30`
Dynamic VLAN (AAA override)	**Enabled.** Without it, everyone lands on the access VLAN.
802.11r Fast BSS Transition	Enabled
OKC (opportunistic key caching)	Enabled
MAC authentication, portals	Off
WLAN group	Add it to the WLAN group the school's APs use

- **Why fast roaming matters:** without 802.11r or OKC, every move between APs is a full login and a Google check. That's slower for the user, and it uses up Google's Secure LDAP quota.
- **Session length:** the server sends a 12-hour session timeout, so a block or an org unit move reaches every device within a school day. If the WLAN has its own session timeout, leave it at its default.
- If a few older devices can't see or join Nexus, try turning 802.11r off and keeping OKC on.

4.5 WLAN: Guest (visitors)

Setting	Value
Name and SSID	`Guest`. It must match `dpsk.ssid` in config.json.
Authentication type, method	Standard usage, **Open**
Encryption	WPA2, AES, with **Dynamic PSK** on (internal DPSK)
DPSK type and length	Secure passphrase, **12 to 16** characters, so a key can be typed off a printed slip
DPSK expiration	**Unlimited.** The platform ends guest keys itself, on the day the admin chose.
Access VLAN	The guest VLAN, for example `50`
Client isolation	On, so visitors' devices can't see each other
Accounting service	`Nexus accounting`, if the vSZ offers it on a DPSK WLAN. Guests then show on the Usage page under their key's name.

Make guest keys in the dashboard, not in the vSZ. Keys that already exist on the vSZ can be brought in with **Settings → Adopt keys**.

4.6 API account for guest keys

1. **Administration → Admins and Roles**. Create a role (a user group) limited to the school's zone, with permission to **manage WLANs**. DPSK keys belong to a WLAN.
2. Create an administrator `wifi-portal-api` in that group, with a long random password. Save the password in the password manager.
3. Don't use a super admin account. If the platform's WLAN list in 5.4 is refused with HTTP 403, the role is too narrow: widen it one step and try again.
4. Open the vSZ web console, click the padlock in the browser, and note the certificate's **SHA-256 fingerprint**. You'll compare it in 5.4.

4.7 School firewall or router

Direction	From	To	Port	For
Out	vSZ	Server IP	UDP 1812, 1813	RADIUS logins and accounting
In (port forward)	**Server IP only**	vSZ management IP	TCP 8443	Guest keys through the vSZ API

Never open 8443 to the whole internet
It's the vSZ's admin API. Limit the port forward to the server's IP. If 8443 is already open to everyone for remote management, restrict it now.

Where RADIUS comes from. With proxy mode and the vSZ at school, RADIUS reaches the server from the school's public IP, after the school's NAT. That's `VSZ_ADDRESS` in part 5, and the address in the `ufw` rule in 2.4. If the vSZ is hosted somewhere else, it's that site's public IP.

RADIUS crosses the internet protected by the shared secret, the server's firewall rule and Message-Authenticator checks. If the school firewall and the server can share a WireGuard or IPsec tunnel, sending RADIUS through it is better still.

PART 5

Finish the server and test

Fill in the two settings files, read the vSZ's certificate fingerprint and IDs, start everything, and log in with a real school account, all before any device joins.

5.1 Permissions

The platform runs as uid 1000 and reads config.json, the service account key and the LDAP certificate. Only Docker, as root, reads `radius.env`.

```
cd /etc/wifi-portal
sudo chown root:1000 config.json service-account.json google-ldap google-ldap/*
sudo chmod 640 config.json service-account.json google-ldap/*
sudo chmod 750 google-ldap
sudo chown root:root radius.env && sudo chmod 600 radius.env
```

5.2 config.json: the platform

Edit it with `sudo nano /etc/wifi-portal/config.json`.

Setting	What to put
`publicUrl`	`https://wifi.phahs.org.za`
`google.clientId`, `clientSecret`	From the OAuth client (3.2)
`google.allowedDomains`	`["phahs.org.za"]`. Personal Gmail and other domains are always turned away.
`google.directory.adminEmail`	A Workspace admin the service account acts as, to read the directory
`radius.apiToken`	The **platform token** from 2.6
`policy.rules`	Org units to VLANs, from the worksheet. They can be changed later on the dashboard's Rules page.
`dpsk.host`	The vSZ address from the worksheet
`dpsk.username`, `password`	The vSZ API account (4.6)
`dpsk.tlsFingerprint`, `zoneId`, `wlanId`	Filled in by 5.4
`dpsk.guestVlanId`	The guest VLAN, for example `50`
`alerts.from`, `alerts.to`	The alert mailbox, and who gets alerts. Delete the whole `alerts` block to turn alerts off.
`backup.drive.folderId`	The Drive folder ID (3.4). Delete the `drive` block if you don't use Drive backups.
`admin.emails`	Who can sign in to the dashboard
`admin.allowFrom`	Optional: IPs or ranges the dashboard accepts, for example the school's public IP and yours. Empty means anywhere, with Google sign-in.
`retentionDays`	How long logins and usage are kept, for POPIA. 90 by default.

5.3 radius.env: FreeRADIUS

Edit it with `sudo nano /etc/wifi-portal/radius.env`.

Setting	What to put
`VSZ_ADDRESS`	The address RADIUS arrives from (4.7), usually the school's public IP
`RADIUS_SECRET`	The **RADIUS shared secret** from 2.6, the same as on the vSZ
`DEFAULT_DOMAIN`	`phahs.org.za`. A bare `t.smith` becomes `t.smith@phahs.org.za`.

Setting	What to put
`LDAP_BASE_DN`	`dc=phahs,dc=org,dc=za`
`PLATFORM_TOKEN`	The **platform token** from 2.6, the same as `radius.apiToken`
`FALLBACK_VLAN`	Your decision from part 1: empty, or a restricted VLAN number
Everything else	Leave as it is

5.4 The vSZ's fingerprint and IDs

Do these in this order. Reading the WLAN list needs the fingerprint first.

```
cd /opt/wifi-portal
sudo docker compose build
sudo docker compose run --rm wifi-portal node scripts/vsz-fingerprint.js /etc/wifi-portal/config.json
```

Compare the fingerprint with the one you noted in 4.6. If they match, paste it into `dpsk.tlsFingerprint`. If they don't, stop: something between the server and the vSZ isn't the vSZ.

```
sudo docker compose run --rm wifi-portal node scripts/vsz-wlans.js /etc/wifi-portal/config.json
```

This lists each zone and WLAN with its ID. Copy the school zone's `zoneId`, and the **Guest** `wlanId`, into config.json.

5.5 Start everything

> **Start it in the evening**
> FreeRADIUS restarts itself every 24 hours, counted from when it started, to pick up renewed certificates. The restart takes a few seconds, and logins in progress during it fail. Starting it after school puts that restart at night.

```
cd /opt/wifi-portal
sudo docker compose up -d
sudo docker compose ps                              # three containers
sudo docker compose logs -f freeradius wifi-portal  # Ctrl+C stops watching
```

- On the very first start, Caddy gets both certificates from Let's Encrypt within a minute, and FreeRADIUS waits for them.
- After a few minutes `docker compose ps` should show all three as **healthy**.
- If `wifi-portal` keeps restarting, `sudo docker compose logs wifi-portal` starts with **Invalid config:** and a list of what to fix in config.json.

5.6 Check the connections

```
sudo docker compose exec wifi-portal node scripts/check-connections.js /etc/wifi-portal/config.json
```

It tries the Google directory, Secure LDAP with the client certificate, Drive, the vSZ API, and sends a test alert email (`--no-email` skips that). Each line says what's wrong if it fails. The dashboard has the same check under **Settings → Check connections**.

5.7 Test a real login, without the vSZ

This logs in to FreeRADIUS on the server the way a phone does, with EAP-TTLS/PAP, and checks the certificate name. It asks for the password, which never appears on the command line. The first run builds a small test image.

```
cd /opt/wifi-portal
sudo ./scripts/test-login.sh a.staff.member@phahs.org.za     # expect: Accepted on VLAN 20
sudo ./scripts/test-login.sh a.learner@phahs.org.za          # expect: Accepted on VLAN 30
sudo ./scripts/test-login.sh a.staff.member@phahs.org.za     # wrong password: expect Refused
```

Every test shows up in the dashboard under **Activity** as device `02:00:00:00:00:99`, with the reason for any refusal.

5.8 Sign in to the dashboard

Open `https://wifi.phahs.org.za/admin` and sign in with an account in `admin.emails`. **Overview** → **Health** shows whether FreeRADIUS is calling, and the state of the Google directory, the vSZ API, backups and alerts.

5.9 The first phone through the vSZ

1. Set up one Android phone as in part 6, and connect it to Nexus with a staff account.
2. In the dashboard, **Activity** shows the login and its VLAN.
3. On the phone, check the IP address is from the staff VLAN's DHCP scope.
4. After 10 to 20 minutes, the phone appears under **Usage** → **Online now**.

If the login is refused or nothing arrives, go to part 8.

PART 6

Devices and the pilot

Each kind of device is set up once, in a way that checks it's talking to the real server. Then run Nexus next to the current network for one or two weeks before everyone moves.

> **Always set devices up this way, never by hand**
>
> Google can only check a plain password, so inside the encrypted tunnel the password is sent as it is (PAP). That's safe only when the device checks it's talking to `radius.phahs.org.za`. A device set up by hand, especially Windows with its "Continue connecting?" prompt, will give the password to a fake access point with the same name. The profiles below, and the Android and Chromebook **Domain** setting, prevent that.

6.1 Making the Apple and Windows profiles

```
cd /opt/wifi-portal
sudo docker compose run --rm wifi-portal node scripts/apple-profile.js /etc/wifi-portal/config.json /var/lib/wifi-portal/Nexus.mobileconfig
sudo docker compose run --rm wifi-portal node scripts/windows-profile.js /etc/wifi-portal/config.json /var/lib/wifi-portal/Nexus.xml
sudo cp /var/lib/wifi-portal/Nexus.* ~ && sudo chown $USER ~/Nexus.*
```

ON YOUR PC (POWERSHELL)

```
scp itadmin@SERVER_IP:~/Nexus.* .
```

Neither file holds a secret, so they can be emailed or put on the intranet.

6.2 Each kind of device

Device	How
iPhone, iPad, Mac	Send `Nexus.mobileconfig` by email or AirDrop and open it. It installs Nexus and asks for the email and password once. Without the profile, Apple devices send the password in a form Google can't check, so they won't connect.
Android	WiFi → Nexus → EAP method **TTLS**, Phase 2 **PAP**, CA certificate **Use system certificates**, Domain `radius.phahs.org.za` (required), Identity: the school email. Leave anonymous identity empty.
Windows 10/11	On each laptop: `netsh wlan add profile filename="Nexus.xml" user=all`, or deploy it with Intune or Group Policy. Then connect and enter the email and password once.
Managed Chromebooks	Admin console → **Devices** → **Networks** → **Wi-Fi**: EAP-TTLS, inner protocol PAP, Server CA **Default**, Subject match / Domain `radius.phahs.org.za`.
Personal Chromebooks	The same settings, typed in by the user.
Very old Android	Without a Domain field, the phone can't check the server. Send it to the Guest network rather than turning checking off.

6.3 Pilot checklist

Run it with IT and a few teachers. Tick each item before telling everyone.

	Check	How
☐	**Every connection works**	**Settings** → **Check connections** is all green, and the test email arrived.
☐	**Google checks passwords**	`test-login.sh` with a staff and a learner account: accepted on their VLANs. With a wrong password: refused.
☐	**The vSZ applies the VLAN**	A staff phone and a learner phone get addresses from their own VLANs. **People** shows the VLAN each login was given.

	Check	How
☐	**Usage arrives**	After 10 to 20 minutes online, the phone shows under **Usage** → **Online now** with growing figures.
☐	**Every kind of device**	iPhone with the profile, a Mac, Android with the Domain set, a Windows laptop with the profile, a school Chromebook and a personal one.
☐	**Turning people away**	Block a test account: its next login is refused, with the reason under **Activity**. Unblock it. Do the same with a suspended test account in Google.
☐	**Roaming**	Walk across the school on a video call: it shouldn't drop, and Activity shouldn't fill with logins.
☐	**Guests**	Make a guest key, print the slip, join the Guest network with it, then end it: the phone can't join again. Note whether the vSZ also drops it at once.
☐	**Fallback**	After school, `sudo docker compose stop wifi-portal`, and check new logins do what you decided in part 1. Then `sudo docker compose start wifi-portal`.
☐	**Backups**	**Settings** → **Back up now**. With Drive set up, the copy appears in the Shared Drive.
☐	**A busy morning**	During the first full morning, watch **Overview** for `directory_error` and slow logins.

6.4 Going live

- Print the connection guide from **Settings** → **Connection guide**. Check its privacy wording against the school's ICT policy first.
- Send the Apple and Windows profiles, and push the network to managed Chromebooks.
- Point an uptime monitor (Uptime Kuma, or your RMM) at `https://wifi.phahs.org.za/healthz`.
- Watch for `directory_error` alerts in the first busy week, and keep fast roaming on.

PART 7

Looking after it

Day to day, the dashboard and the alert emails tell you when something needs attention. This part covers the jobs around them.

7.1 Updating the platform

After school, from `/opt/wifi-portal`:

```
git pull
sudo docker compose up -d --build
sudo docker compose ps
```

Only containers whose code changed are restarted. Connected devices stay connected while the platform restarts.

7.2 Server updates and reboots

- Security updates install themselves (2.2). Run `sudo apt update && sudo apt full-upgrade` now and then for everything else.
- When `/var/run/reboot-required` exists, reboot after school or in the holidays with `sudo reboot`. All three containers come back by themselves.

7.3 Backups: what's covered and what isn't

What	Where it's kept
The database: logins, people, usage, guest keys, rules	Copied every night to `/var/lib/wifi-portal/backups/` (the last 14), and to the Drive folder if set up (the last 30, gzipped).
Settings and secrets in `/etc/wifi-portal`	**Not backed up automatically.** Make a copy after setup and after each change, as below.
The code	On GitHub: `github.com/pha-llotter/wifi-platform`
Let's Encrypt certificates	Not needed: Caddy gets new ones by itself.

```
sudo tar czf ~/wifi-portal-etc-$(date +%F).tgz -C /etc wifi-portal
sudo chown $USER ~/wifi-portal-etc-*.tgz      # then copy it to your PC with scp
```

That file holds every secret. Store it encrypted, for example as an attachment in the password manager, and delete it from the server.

7.4 Restoring the database

```
cd /opt/wifi-portal
sudo docker compose stop wifi-portal
sudo mkdir -p /var/lib/wifi-portal/before-restore
sudo mv /var/lib/wifi-portal/wifi-portal.db* /var/lib/wifi-portal/before-restore/
sudo cp /var/lib/wifi-portal/backups/wifi-portal-2026-11-20-00-00-00.db /var/lib/wifi-portal/wifi-portal.db
sudo chown 1000 /var/lib/wifi-portal/wifi-portal.db
sudo docker compose start wifi-portal
```

- Moving `wifi-portal.db*` aside also moves the database's `-wal` and `-shm` files. Left in place, they would be mixed into the restored copy.
- For a copy from Drive, run `gunzip` on it first.
- Then use **Settings** → **Adopt keys** to bring back guest keys made since that copy.

7.5 Rebuilding a lost server

1. Do 2.1 to 2.5 on a new server. If the IP changed, update both DNS records, the vSZ RADIUS services, the school's port forward and the `ufw` rules.
2. Put back `/etc/wifi-portal` from your copy: `sudo tar xzf wifi-portal-etc-DATE.tgz -C /etc`. Skip 2.6: it would overwrite the restored settings with the examples.
3. Make the data folders:
   `sudo mkdir -p /var/log/wifi-portal /var/lib/wifi-portal && sudo chown 1000 /var/log/wifi-portal /var/lib/wifi-portal`.
   Then restore the latest database (7.4) into `/var/lib/wifi-portal`.
4. Start as in 5.5, then check as in 5.6 and 5.7.

7.6 Things that expire or change

What	What to do
Google Secure LDAP certificate	Before it expires, generate a new one on the LDAP client, copy it to `/etc/wifi-portal/google-ldap/` with the same names and permissions, then `sudo docker compose restart freeradius` after school.
The vSZ's own certificate	If it's ever replaced, guest keys stop working with a fingerprint error. Run 5.4 again, compare, and update `dpsk.tlsFingerprint`.
Let's Encrypt	Automatic. Caddy renews, and FreeRADIUS picks it up at its daily restart.
IT staff changes	Update `admin.emails` and `alerts.to`, then `sudo docker compose restart wifi-portal`.
Changing the RADIUS secret	Change it on both vSZ services and in `radius.env` at the same time, after school, then `sudo docker compose up -d freeradius`.
Changing the platform token	Change `radius.apiToken` and `PLATFORM_TOKEN` together, then `sudo docker compose up -d --force-recreate`.

7.7 Logs

```
sudo docker compose logs --tail 200 freeradius      # RADIUS: logins and refusals
sudo docker compose logs --tail 200 wifi-portal     # the platform
sudo tail -f /var/log/wifi-portal/wifi.jsonl        # every login and admin action
```

Docker keeps 5 × 20 MB of output per container. `wifi.jsonl` rotates at 50 MB and keeps 5 old copies.

PART 8

When something doesn't work

Start with the dashboard: **Activity** gives the reason for every refusal, and **Overview** → **Health** shows which connection is down. The table below covers the rest.

8.1 Seeing every login in detail

```
sudo nano /etc/wifi-portal/radius.env      # set RADIUS_DEBUG=1
sudo docker compose up -d freeradius
sudo docker compose logs -f freeradius     # try a login, then Ctrl+C
```

Debug output shows passwords

In debug mode FreeRADIUS prints each login in full, including the password from inside the tunnel. Use it with test accounts, for as short a time as you can, then set `RADIUS_DEBUG=0` and run `sudo docker compose up -d freeradius` again.

8.2 Symptoms

You see	Likely cause	What to do
FreeRADIUS log: *Ignoring request ... from unknown client 198.51.100.7*	RADIUS arrives from a different address than `VSZ_ADDRESS`	Set `VSZ_ADDRESS` to the address in the message, fix the `ufw` rule to match, then `sudo docker compose up -d freeradius`.
Nothing in the FreeRADIUS log at all	A firewall in the way: the server's, the provider's, or the school's	On the server, `sudo tcpdump -ni any udp port 1812` while a phone tries. No packets means it's being blocked before the server.
The vSZ's "Test AAA" fails	Expected: it isn't an 802.1X login	Use `test-login.sh` (5.7) instead.
`test-login.sh`: refused with a correct password; Activity says wrong password	Secure LDAP: the client is off, a permission is missing, or the user's org unit isn't included	Check 3.1. Run **Check connections**: its LDAP line says what Google answered.
Check connections: LDAP can bind but can't search, or Google asks for credentials	This LDAP client needs access credentials as well as the certificate	Admin console → the LDAP client → **Access credentials** → **Generate new credentials**. Add `LDAP_IDENTITY=` and `LDAP_PASSWORD=` lines with them to radius.env, then `sudo docker compose up -d freeradius`.
LDAP: a TLS or certificate error	The server's clock is wrong, or the LDAP certificate files don't match	`timedatectl` should show *synchronized: yes*. Download the certificate again (3.1).
Every login refused; Activity empty; Health says FreeRADIUS hasn't called	The platform token differs between config.json and radius.env	Make `radius.apiToken` and `PLATFORM_TOKEN` the same, then `sudo docker compose up -d --force-recreate`.
Accepted, but everyone is on the same VLAN	Dynamic VLAN (AAA override) is off on Nexus	Turn it on (4.4). People and Activity show which VLAN the server chose.
Accepted, but the device gets no IP address	That VLAN doesn't reach the AP, or has no DHCP scope	Check the switch port is tagged for the VLAN, and the DHCP scope.
Refused: rule, blocked, suspended or wrong domain	Working as intended	Activity gives the reason. Change the rule, the block or the Google account.
An iPhone or Mac won't connect	Joined by hand, without the profile	Forget the network, install the profile (6.1).
Windows asks "Continue connecting?"	Set up by hand	Don't continue. Install the profile (6.2).
`wifi-portal` keeps restarting	A mistake in config.json	`sudo docker compose logs wifi-portal` lists each problem under *Invalid config*.
Guest keys: "doesn't match the pinned fingerprint"	The vSZ certificate changed, or something is intercepting	Run 5.4 again and compare with the vSZ console before updating.

You see	Likely cause	What to do
Guest keys: sign-in failed or HTTP 403	The API account's password or role	Check the password, then widen the role (4.6).
Guest keys: can't reach the vSZ	The school's port forward for TCP 8443	Check 4.7, including that it allows the server's IP.
`directory_error` alerts	Google directory or quota trouble, or delegation scopes	Check the delegation entry (3.3). On busy days, check fast roaming is on (4.4).

APPENDIX

Ports and files

Ports

On	Port	Open to	For
Server, in	TCP 22	Your IPs	SSH
Server, in	TCP 80, 443	Anywhere	Dashboard, Let's Encrypt
Server, in	UDP 1812, 1813	The RADIUS source IP only	Logins and accounting from the vSZ
Server, out	TCP 636	ldap.google.com	Password checks
Server, out	TCP 443	Google APIs, Let's Encrypt, Docker Hub, GitHub	Directory, alerts, backups, certificates, updates
Server, out	TCP 8443	The vSZ	Guest keys
Server, local only	TCP 8080; UDP 18120, 18121	127.0.0.1 only: never open	Platform API; FreeRADIUS inner tunnel and health check
School, out	UDP 1812, 1813	The server	RADIUS from the vSZ
School, in (port forward)	TCP 8443	The server's IP only	The vSZ API

Files on the server

Path	What it is
`/opt/wifi-portal`	The code. `docker compose` commands run from here.
`/etc/wifi-portal/config.json`	Platform settings (root:1000, 640)
`/etc/wifi-portal/radius.env`	FreeRADIUS settings and secrets (root, 600)
`/etc/wifi-portal/service-account.json`	Google service account key (root:1000, 640)
`/etc/wifi-portal/google-ldap/ldap-client.crt`, `.key`	Google Secure LDAP client certificate (root:1000, 640)
`/var/lib/wifi-portal/wifi-portal.db`	The database
`/var/lib/wifi-portal/backups/`	Nightly copies, the last 14
`/var/log/wifi-portal/wifi.jsonl`	Logins and admin actions, one JSON line each
Docker volume `caddy-data`	Let's Encrypt certificates. FreeRADIUS reads the RADIUS name's certificate from here.

Containers

Name	Does
`caddy`	HTTPS for the dashboard, and the certificate for `radius.phahs.org.za`
`wifi-portal`	The platform: the VLAN decision, the dashboard, guest keys, usage, alerts, backups
`freeradius`	802.1X logins for Nexus, checked against Google Secure LDAP

The repo's README has the same setup in shorter form, plus how the rules, usage figures and tests work.